JAVA INTERVIEW PREPARATION

CHAPTER 25

COMPLETABLEFUTURE

Senior / Lead Java Developer Textbook Edition

Full reading material - not summarized

CompletableFuture and Asynchronous Composition: Building Non-Blocking Dependency Graphs

Senior / Lead Java Developer Reading Chapter

CompletableFuture represents a value and a graph of actions that should run after completion. Its strength is composition: transform a result, start a dependent operation, combine independent results, recover from failure, and apply deadlines without manually coordinating callbacks. Its danger is hidden execution policy, accidental blocking, confusing exception stages, and background work that outlives the request that created it.

A Stage Graph, Not Merely a Future

```
load customer ----\
                   +--> combine --> price --> response
load orders ------/
```

Each method creates a new stage. The original future is not modified.

```
CompletableFuture<Customer> customer =
    CompletableFuture.supplyAsync(() -> customerClient.load(id), ioExecutor);

CompletableFuture<CustomerView> view =
    customer.thenApply(this::toView);
```

CompletableFuture is also a Future, so join and get can block. Good composition delays blocking until a clearly chosen boundary or returns the stage to a framework that understands asynchronous completion.

Execution Policy: Async Is Not One Thing

Non-Async continuation methods such as thenApply may run in the thread that completes the previous stage, or immediately in the caller if already complete. Async variants schedule through an executor.

```
future.thenApply(this::transform);
future.thenApplyAsync(this::transform, cpuExecutor);
```

Async overloads without an executor generally use the ForkJoin common pool. That global pool is shared by unrelated work and is primarily designed for CPU-oriented fork/join tasks. Blocking database or HTTP calls there can consume its workers and interfere with other components.

Pass an explicitly owned executor when workload isolation, blocking behavior, context, or operational control matters.

thenApply Versus thenCompose

thenApply transforms a value synchronously and wraps the transformed result in a stage.

```
CompletableFuture<String> name = customer.thenApply(Customer::name);
```

If the transformation itself returns a CompletionStage, thenApply creates nesting:

```
CompletableFuture<CompletableFuture<Orders>> nested =
    customer.thenApply(c -> loadOrdersAsync(c.id()));
```

thenCompose flattens the dependent asynchronous operation:

```
CompletableFuture<Orders> orders =
    customer.thenCompose(c -> loadOrdersAsync(c.id()));
```

The conceptual distinction matches map versus flatMap: thenApply transforms T to U; thenCompose chains T to CompletionStage<U>.

Combining Independent Work

thenCombine joins two independent results without blocking a worker merely to call get.

```
CompletableFuture<Customer> customer = loadCustomer(id);
CompletableFuture<List<Order>> orders = loadOrders(id);

CompletableFuture<CustomerPage> page = customer.thenCombine(
    orders,
    CustomerPage::new
);
```

Start independent operations before combining them. Writing loadCustomer(id).thenCompose(c -> loadOrders(id)) serializes calls that could have overlapped.

Whether parallel dependency calls improve latency depends on downstream capacity and failure behavior. Fan-out increases concurrent load and can amplify a slow dependency.

allOf and Typed Result Collection

allOf completes when every supplied future completes, but its result type is Void. The original futures still hold typed results.

```
List<CompletableFuture<Item>> futures = ids.stream()
    .map(this::loadItem)
    .toList();

CompletableFuture<List<Item>> all = CompletableFuture
    .allOf(futures.toArray(CompletableFuture[]::new))
    .thenApply(ignored -> futures.stream()
        .map(CompletableFuture::join)
        .toList());
```

The joins occur after allOf succeeds, so they do not normally wait. If one stage fails, allOf completes exceptionally, but other operations are not automatically cancelled. Define whether to collect partial results, fail fast, cancel siblings, or wait for all diagnostics.

Failure Is Part of the Graph

Exceptions thrown by a supplier or continuation complete that stage exceptionally. Dependent success-only stages are skipped until a recovery or observation stage handles the failure.

```
CompletableFuture<Price> price = loadPrice(id)
    .exceptionally(error -> fallbackPrice(error));
```

`exceptionally` recovers only from failure and produces a replacement value. `handle` receives either result or error and always transforms. `whenComplete` observes success or failure but normally preserves the

original outcome unless its own action fails.

```
future.whenComplete((result, error) -> {
    if (error != null) metrics.recordFailure(error);
});
```

CompletionException and ExecutionException wrap underlying causes at join and get boundaries. Preserve the root cause and domain meaning when translating failures.

Recovery should be selective. Returning an empty list for authentication failure, data corruption, and a harmless missing recommendation treats unlike failures as equal and can create false success.

Timeouts and Deadlines

`orTimeout` completes a future exceptionally after the timeout. `completeOnTimeout` supplies a fallback value.

```
CompletableFuture<Quote> quote = loadQuote(id)
    .orTimeout(800, TimeUnit.MILLISECONDS);
```

Completing the CompletableFuture exceptionally does not guarantee the underlying HTTP call, JDBC statement, or arbitrary supplier stops. Configure timeouts in the dependency client and propagate cancellation where supported.

An overall request deadline should be divided across children. Independent fixed timeouts at every layer can exceed the caller's remaining budget. Capture remaining time and refuse work that cannot finish meaningfully.

Cancellation Semantics

Cancelling a CompletableFuture marks it cancelled and causes dependent stages to fail accordingly, but it does not offer the same guaranteed interrupt attempt developers may expect from an executor-backed Future. The underlying action needs an explicit cancellation mechanism if stopping it matters.

Cancellation is a graph design question:

```
parent no longer needs result
       |
       +--> mark stage cancelled
       +--> cancel HTTP/JDBC operation if supported
       +--> prevent later side effects
       +--> release permits and context
```

Never assume a client timeout means background work cannot commit a write later. Use idempotency and transaction design for side effects.

Avoid Blocking Inside Stages

This pattern consumes an executor worker while waiting for work that could have been composed:

```
thenApply(value -> anotherFuture(value).join()) // hidden blocking
```

Use thenCompose. Blocking can be reasonable at a boundary such as a command-line main method, but blocking within a small shared pool can cause starvation or deadlock.

Also avoid performing long CPU work in a completion thread that should quickly release I/O resources. Move significant CPU transformation to an appropriate bounded executor with an Async method.

Fan-Out Needs a Bound

Creating one future for each of a million records starts a million logical operations and retains a large graph.

```
List<CompletableFuture<Result>> results = ids.stream()
    .map(this::callRemoteAsync)
    .toList();
```

Executor size may limit active threads, but the queue and future graph can still be enormous. Process bounded batches, use a Semaphore around dependency calls, use a bounded executor queue, or adopt a streaming system with backpressure.

Context and Transactions

ThreadLocal logging, security, tracing, and transaction context do not automatically follow a stage onto another executor. Capture only the required immutable context or use framework-supported propagation.

A Spring transaction is normally thread-bound. Starting supplyAsync inside a transactional method does not automatically extend that transaction to the executor thread. Design each asynchronous unit with its own explicit transaction boundary or keep transactional work on the owning thread.

```
request thread transaction
       |
       +-- async executor thread: different thread, no automatic transaction
```

This is a frequent production source of LazyInitializationException, missing security identity, and incomplete tracing.

CompletableFuture and Virtual Threads

CompletableFuture is valuable for explicit dependency graphs and APIs already modeled as CompletionStage. Virtual threads make straightforward blocking composition scalable for many I/O-heavy request paths.

```
Customer customer = customerClient.load(id);
Orders orders = orderClient.load(id);
```

Sequential blocking code is simpler but does not overlap independent calls unless tasks are forked. CompletableFuture expresses overlap but increases graph and context complexity. Choose based on concurrency needs, ecosystem, error semantics, observability, and team maintainability rather than fashion.

Testing Asynchronous Graphs

Inject executors rather than hiding global pools. Unit tests can use a direct executor for deterministic simple stages, while integration tests exercise real concurrency. Test success, each dependency failure, timeout, cancellation, executor rejection, partial completion, context propagation, and shutdown.

Avoid arbitrary sleeps. Control completion with test futures, latches, fake clocks where applicable, and bounded await assertions. A test that passes only because a machine happened to schedule quickly is not reliable.

Production Perspective

Common incidents include blocking the common pool with JDBC, losing trace context, serializing supposedly parallel calls, unbounded fan-out, timeouts that leave work running, and broad exceptionally handlers turning outages into plausible but wrong data. Another frequent issue is a controller calling join, eliminating non-blocking benefits and tying up request threads.

Observe stage and dependency latency, executor active and queue metrics, rejection, timeout and cancellation, graph cardinality, common-pool utilization, downstream concurrency, and abandoned work after caller timeout.

Interview-Ready Summary

CompletableFuture models a value plus a graph of dependent stages. thenApply transforms a value, thenCompose flattens a dependent asynchronous operation, thenCombine combines independent results, and allOf coordinates many stages without producing typed results itself.

Non-Async and Async variants have different execution behavior. Avoid accidental use of the common pool for blocking work, hidden joins inside stages, and unbounded fan-out. Model failure, timeouts, cancellation, context, and transaction boundaries explicitly. CompletableFuture completion does not automatically stop underlying work.

Revision Notes

- CompletableFuture represents both completion and a stage graph.
- Every transformation returns a new stage.
- Non-Async continuations may run in the completing thread.
- Async overloads without an executor commonly use the common pool.
- Do not place uncontrolled blocking I/O on the common pool.
- thenApply maps T to U.
- thenCompose chains T to CompletionStage<U> and flattens it.
- thenCombine combines independent results.
- allOf returns Void; typed results remain in the original futures.
- allOf does not automatically cancel sibling work after failure.
- exceptionally recovers from failure.
- handle transforms either success or failure.
- whenComplete observes while normally preserving the outcome.
- Preserve root causes when unwrapping completion exceptions.
- Timeout of a stage does not guarantee underlying work stops.
- Cancellation requires cooperation from the actual operation.
- Avoid join inside stages when composition is possible.
- Bound fan-out, queues, and downstream concurrency.
- ThreadLocal and transaction context do not automatically cross executors.
- Virtual threads and CompletableFuture solve overlapping but different concerns.
- Test failure, timeout, cancellation, rejection, context, and shutdown deterministically.